BrahmaputraOS / Luit

A Student's GDB Practice Session

System Calls · Page Tables · The Process Table

Everything you need to understand and complete Lab 2 and Lab 3

CS3106L — Operating Systems Laboratory

Dr. Satyajit Das

Department of Computer Science and Engineering · IIT Guwahati

0. How to Use This Session

This is a hands-on session, not a reference to skim. Open two terminals side by side, boot Luit under the debugger, and type every command yourself. Each command below is followed by what you will actually see and — more importantly — why it matters. The goal is that by the end you can find, on your own, the exact line in the Luit kernel where a system call is dispatched, where a page is mapped, and where the process table records a running program. Those three skills are precisely what Lab 2 and Lab 3 require.

> **What you will be able to do by the end**
>
> Set a breakpoint at the single point every system call crosses, and read its number and arguments.
> Walk a page table by hand and confirm why the Lab 3 info page is read-only in user space.
> Print the process table and read the Lab 2 and Lab 3 fields you are about to implement.

0.1 Prerequisites

- A working Luit checkout that builds: `make` produces `kernel.elf` and `fs.img`.
- The RISC-V–aware GDB: `gdb-multiarch` (Debian/Ubuntu) or `riscv64-unknown-elf-gdb`.
- The kernel is already compiled with debug symbols — the Makefile passes `-ggdb -gdwarf-2`, so no rebuild flags are needed.

0.2 One-time GDB safe-path setup

Luit ships a `.gdbinit` in the checkout root that connects to QEMU and loads helper commands. GDB will refuse to auto-load a `.gdbinit` from a project directory unless you allow it once. Add this line to `~/.config/gdb/gdbinit` (create the file if needed):

```
add-auto-load-safe-path /path/to/your/luit/.gdbinit
```

> **On the shared lab machines**
>
> The safe-path has already been configured for /home by the lab administrators, so on the department machines you can skip this step. If GDB prints a warning about auto-loading being declined, this is the setting that fixes it.

1. Boot Luit Under the Debugger

In **terminal 1**, start Luit paused, waiting for the debugger:

```
$ make qemu-gdb
```

This runs QEMU with `-S -gdb tcp::1234`: the `-S` freezes the machine before the first instruction, and `-gdb tcp::1234` opens a debug port. Nothing else will happen until you attach.

In **terminal 2**, from the same checkout directory, launch GDB:

```
$ gdb-multiarch kernel.elf
```

The shipped `.gdbinit` automatically sets the architecture, connects to `localhost:1234`, loads the symbols, and prints:

```
[Luit] connected. Helpers: trapregs, procs, tf <ptr>.
[Luit] Suggested first breakpoints: b usertrap / b syscall / b scheduler
```

The three helper commands the .gdbinit gives you

trapregs — dump the trap CSRs (scause, sepc, stval, sstatus, satp) the instant you stop in a trap handler.
procs — walk the process table and print every slot that is not UNUSED.
tf <ptr> — print the saved user registers of a trapframe.

A note on the tf helper's help text

If you run 'help tf' the description mentions myproc()->trapframe. In the current Luit source the field is actually named tf, so the correct call is: tf myproc()->tf . Use that spelling; the helper body itself is correct.

2. Your First Breakpoint: the Scheduler

Let the machine run until the kernel is alive and scheduling processes. Set a breakpoint on the scheduler and continue:

```
(gdb) b scheduler
(gdb) c
```

When it stops, you are inside `scheduler()` in `kernel/proc.c`, the loop that hands each runnable process a turn on the CPU. Confirm where you are:

```
(gdb) bt
(gdb) list
```

`bt` (backtrace) shows the call stack; `list` shows the source around the current line. You have now proven the kernel booted and reached its main loop. Everything after this is about watching user programs cross into the kernel and back.

3. System Calls — the Heart of Lab 2 and Lab 3

Both labs hook into the exact same place: the single function every system call passes through. In Luit that function is `syscall()` in `kernel/syscall.c`. Read it once before you break on it:

3.1 What the code does

```
void syscall(void)
{
    struct proc *p = myproc();
    uint64 num = p->tf->a7;          // the syscall number arrives in a7

    if (num > 0 && num < NSYS && syscalls[num]) {
        uint64 a0 = p->tf->a0;       // first argument (saved for tracing)
        p->tf->a0 = syscalls[num](); // DISPATCH: call the handler, store return
        trace_record((int)num, ...); //  <-- Lab 2 hooks in HERE
        p->syscall_count++;          //  <-- Lab 3 counts HERE
        usysinfo_update(p);          //  <-- Lab 3 refreshes the shared page
HERE
    } else {
        p->tf->a0 = -1;              // unknown number: fail, stay alive
    }
}
```

Read this twice — it is the whole point

The RISC-V calling convention puts the system-call NUMBER in register a7 and the ARGUMENTS in a0, a1, a2, … The return value is written back into a0.
This one function is the choke point every call crosses. That is exactly why Lab 2 (tracing) and Lab 3 (the counter in the shared page) both attach here — one place to see every call.

3.2 Break on it and read a live call

Set the breakpoint and continue. In terminal 1, type a command at the Luit shell (for example, run a program). The debugger will stop the moment the kernel enters syscall():

```
(gdb) b syscall
(gdb) c
```

When it stops, read the number and the first argument straight from the trapframe:

```
(gdb) p p->tf->a7        # the syscall number
(gdb) p p->tf->a0        # the first argument
(gdb) p p->name          # which process made the call
```

Cross-check the number against the table. Luit's system calls are generated from `kernel/syscall.tbl`. The numbers you will see most often early in boot:

a7	Call	What it does
1	fork	create a child process
5	read	read from a file descriptor
7	exec	replace the program image
16	write	write to a file descriptor
11	getpid	return the caller's pid
24	tracectl	Lab 2: control tracing

Step through the dispatch to watch the handler run and the return value land in `a0`:

```
(gdb) next              # step over, line by line
(gdb) next
(gdb) p p->tf->a0       # after dispatch: this is the return value
```

Connecting this to Lab 2

trace_record(num, ret, arg0) is called right after dispatch with exactly the values you just printed by hand. Your Lab 2 job is to store those values into a per-hart ring buffer. Put a breakpoint on trace_record and you are standing inside the data your tracer must capture.

```
(gdb) b trace_record
(gdb) c
(gdb) p num
(gdb) p ret
(gdb) p arg0
```

4. Page Tables — Why the Lab 3 Page Is Read-Only

Lab 3 maps a shared information page into every process at a fixed virtual address and requires it to be **read-only in user space**. GDB lets you prove that mapping exists and carries exactly the right permission bits. First, the facts baked into the Luit source:

Fact	Value	Where
Info-page virtual address	`0x7FFFF000`	kernel/usysinfo.h
Permissions requested	`PTE_R | PTE_U`	usysinfo.c (mappages call)
Page size	`4096 bytes (PGSHIFT=12)`	kernel/riscv.h
The walk function	`walk(pt, va, alloc)`	kernel/vm.c

The bit values you will check against (from `kernel/riscv.h`): `PTE_V=1` (valid), `PTE_R=2` (readable), `PTE_W=4` (writable), `PTE_U=16` (user-accessible).

4.1 Find a process and its page table

Break where a user process is definitely running, then get the current process and its page table root:

```
(gdb) b syscall
(gdb) c
(gdb) p p->pagetable        # the root page-table pointer for this process
(gdb) p/x p->sz             # the size of its user address space
```

4.2 Walk to the info page by hand

The RISC-V Sv39 scheme splits a virtual address into three 9-bit indices plus a 12-bit offset. Rather than decode it on paper, let Luit's own `walk()` do it. Call it from GDB on the info-page address:

```
(gdb) p/x walk(p->pagetable, 0x7FFFF000, 0)   # 0 = do not allocate
```

That returns the address of the **page-table entry** (PTE) for the info page. Dereference it to see the raw entry, then read its permission bits:

```
(gdb) set $pte = *(unsigned long *)walk(p->pagetable, 0x7FFFF000, 0)
(gdb) p/x $pte
(gdb) p ($pte & 0x1)  != 0    # PTE_V  valid?     -> expect 1
(gdb) p ($pte & 0x2)  != 0    # PTE_R  readable?  -> expect 1
(gdb) p ($pte & 0x4)  != 0    # PTE_W  writable?  -> expect 0  (READ-ONLY!)
(gdb) p ($pte & 0x10) != 0    # PTE_U  user?      -> expect 1
```

This is the Lab 3 safety property, proven in the debugger

PTE_W is 0: user code physically cannot write the info page — a write faults the process, and the kernel's copy stays safe. PTE_U is 1: user code may read it without trapping, which is the whole speed advantage over a system call. If you ever map it writable by mistake, this is exactly the check that will catch it.

4.3 Turn a PTE into a physical address

The top bits of the PTE (above bit 10) are the physical page number. Shift and recombine to get the physical address the info page actually lives at:

```
(gdb) p/x (($pte >> 10) << 12)   # PTE -> physical address of the page
```

You have now followed a virtual address all the way to physical memory, and confirmed the exact permissions Lab 3 depends on — without writing a line of kernel code.

5. The Process Table — Reading the Lab 2 and Lab 3 Fields

Every process lives in a slot of the global array `proc[]` in `kernel/proc.c`. The shipped helper walks it for you:

```
(gdb) procs
```

You will see one line per live process — pid, state, and name. The state numbers come from enum procstate:

Value	State	Meaning
0	UNUSED	empty slot
1	USED	being set up
2	SLEEPING	blocked on a channel
3	RUNNABLE	ready, waiting for a CPU
4	RUNNING	on a CPU now
5	ZOMBIE	exited, not yet reaped

5.1 Inspect the fields you are about to implement

Pick a running process (say the current `p`) and print the Lab 2 and Lab 3 members of its `struct proc` directly:

```
(gdb) p p->pid
(gdb) p p->trace_filter      # Lab 2: which syscalls this process traces
(gdb) p p->syscall_count     # Lab 3: running count of syscalls made
(gdb) p p->ctxsw_count       # Lab 3: context-switch count
(gdb) p p->usysinfo          # Lab 3: pointer to this process's info page
```

Why this matters before you write any code

These fields already exist in struct proc (see kernel/defs.h). Lab 2 fills trace_filter and feeds a ring buffer; Lab 3 keeps syscall_count / ctxsw_count / state_gen current and mirrors them into the usysinfo page. Watching them here — before implementing — shows you exactly what your code must keep accurate.

6. Guided Sequence for Lab 2 (Kernel Tracing)

Now put it together. This sequence walks the exact path your tracing subsystem must instrument.

Step 1 — See the choke point fire for every call

```
(gdb) b syscall
(gdb) c
```

Each `c` (continue) stops at the next system call. Confirm the number changes as different calls arrive: `p p->tf->a7`.

Step 2 — Watch the record hook

```
(gdb) b trace_record
(gdb) c
(gdb) p num
(gdb) p ret
(gdb) p arg0
(gdb) bt
```

The backtrace shows `trace_record` is called from `syscall` — the one place you needed. The three values are exactly what one ring-buffer entry must hold.

Step 3 — Find the per-hart ring

Lab 2 keeps a separate ring per hart to avoid lock contention. Inspect the ring state (the tracer stores `head`, `count`, and a `dropped` overflow counter):

```
(gdb) p rings[0].head
(gdb) p rings[0].count
(gdb) p rings[0].dropped     # increments when the ring is full - overflow,
never silent
```

Step 4 — Confirm inheritance across fork

A traced process's children are traced too. Break in `fork` and confirm the child copies the parent's `trace_filter`:

```
(gdb) b fork
(gdb) c
(gdb) # after the child proc np is allocated:
(gdb) p np->trace_filter
```

> **What 'trace ls' shows you from userland**
>
> Back in terminal 1 you can run the tracer end-to-end. A real session prints lines like: pid 4 hart 0 fork(arg0=0) = 5 and ends with '--- 23 trace events ---'. Every one of those lines is a ring entry your Step-2 breakpoint watched being created.

7. Guided Sequence for Lab 3 (Versioned Shared Info Page)

This sequence walks the seqlock protocol and the read-only mapping that make Lab 3 correct.

Step 1 — Confirm the mapping and its permissions

Use the exact page-table walk from Section 4 to prove the page is present, readable, user-accessible, and NOT writable:

```
(gdb) b syscall
(gdb) c
```

```
(gdb) set $pte = *(unsigned long *)walk(p->pagetable, 0x7FFFF000, 0)
(gdb) p ($pte & 0x4) != 0    # PTE_W must be 0
```

Step 2 — Watch the seqlock writer

The kernel updates the page under a seqlock: it bumps `seq` to an odd number, writes the fields, then bumps `seq` to even. Break in the updater and watch `seq` step:

```
(gdb) b usysinfo_update
(gdb) c
(gdb) p p->usysinfo->seq     # note the value
(gdb) next
(gdb) next
(gdb) next
(gdb) p p->usysinfo->seq     # it has advanced by 2 (odd -> even)
```

> **The rule the seqlock enforces**
>
> seq odd = an update is in progress (readers must retry). seq even = a stable snapshot. The two __sync_synchronize() memory barriers around the field writes are what stop the compiler or CPU from reordering the bump past the data. Drop a barrier and a reader can see a half-updated page — that is the bug the torn-snapshot test hunts for.

Step 3 — Read the fields the user side will see

```
(gdb) p p->usysinfo->pid
(gdb) p p->usysinfo->syscall_count
(gdb) p p->usysinfo->ticks
(gdb) p p->usysinfo->version    # must equal USYSINFO_VERSION (1)
```

Step 4 — See the userland reader retry loop

On the user side, `u_snapshot()` in `user/ulib.c` reads `seq`, copies the fields, then re-reads `seq`; if it changed, it loops. That is the reader half of the seqlock. Running `usitest` from terminal 1 exercises it 20,000 times and asserts `torn=0`. A real run prints:

```
u_getpid=3 getpid=3 OK
consistency: 20000 snapshots, torn=0 OK
syscall_count 42 -> 47 OK
```

The `torn=0` line is your correctness proof: across twenty thousand reads while the kernel was updating the page, the reader never once saw an inconsistent snapshot.

8. One-Page GDB Cheat Sheet

Command	What it does
`make qemu-gdb`	boot Luit paused, waiting for the debugger (terminal 1)
`gdb-multiarch kernel.elf`	attach the debugger (terminal 2)
`b syscall`	break at the system-call choke point
`b usertrap / b scheduler`	break at trap entry / the scheduler loop
`c`	continue until the next breakpoint
`next / step`	step over / step into one source line
`bt`	backtrace: the current call stack
`list`	show source around the current line
`p expr`	print a C expression (e.g. p p->tf->a7)
`p/x expr`	print in hexadecimal
`trapregs`	dump scause/sepc/stval/sstatus/satp (from .gdbinit)
`procs`	print every live process-table slot (from .gdbinit)
`tf myproc()->tf`	print the saved user registers (from .gdbinit)
`walk(pt, va, 0)`	get the PTE address for a virtual address

Key addresses and numbers to remember: the info page lives at `0x7FFFF000`; the syscall number is in `a7`; `PTE_W` is bit value `4` and must be `0` for the Lab 3 page; kernel RAM begins at `0x80000000`.

— End of practice session —